**National University of Computer & Emerging Sciences, Karachi**
**Computer Science Department**
**Spring 2026, Lab Manual – 07**

| | |
|---|---|
| **Course Code: CL-1004** | **Course: Object Oriented Programming Lab** |
| **Instructor :** | **Muhammad Owais** |

# LAB - 7

# Type Casting and Inheritance

# CASTING

Type Casting is a feature in Java using which the form or type of a variable or object is cast into some other kind or Object, and the process of conversion from one type to another is called Type Casting.

In Java, there are two types of casting:

Widening Casting (automatically) - converting a smaller type to a larger type size

byte -> short -> char -> int -> long -> float -> double

Narrowing Casting (manually) - converting a larger type to a smaller size type

double -> float -> long -> int -> char -> short -> byte

## Implicit Type Casting

In **Implicit Type Casting** also known as **Widening Type Casting**, Java automatically converts one data type to another data type.

Example:

```
public class implicitcasting {
    public static void main(String [] args){
     // create int type variable
    int num = 10;
    System.out.println("The integer value: " + num);

    // convert into double type
    double data = num;
    System.out.println("The double value: " + data);
  }

}
```

Output:

```
The integer value: 10
The double value: 10.0
```

Java first converts the int type data into the double type. And then assign it to the double variable. The lower data type (having smaller size) is converted into the higher data type (having larger size). Hence there is no loss in data. This is why this type of conversion happens automatically.

## Explicit Type Casting

In **Explicit Type Casting** also known as **Narrowing Type Casting**, we manually convert one data type into another using the parentheses.

Example:

```
public class explicitcasting {
    public static void main(String[] args) {
    // create double type variable
    double num = 10.99;
    System.out.println("The double value: " + num);

    // convert into int type
    int data = (int)num;
    System.out.println("The integer value: " + data);
  }

}
```

Output:

```
The double value: 10.99
The integer value: 10
```

In the above example, we are assigning the double type variable named num to an int type variable named data.

Notice the line,

**int data = (int)num;**

Here, the int keyword inside the parentheses indicates that that the num variable is converted into the int type.

In the case of Narrowing Type Casting, the higher data types (having larger size) are converted into lower data types (having smaller size). Hence there is the loss of data. This is why this type of conversion does not happen automatically.

**Example: Type conversion from int to String**

```
public class explicitcasting {
public static void main(String[] args) {
    // create int type variable
    int num = 10;
    System.out.println("The integer value is: " + num);

    // converts int to string type
    String data = String.valueOf(num);
    System.out.println("The string value is: " + data);
  }
}
```

Output:

```
The integer value is: 10
The string value is: 10
```

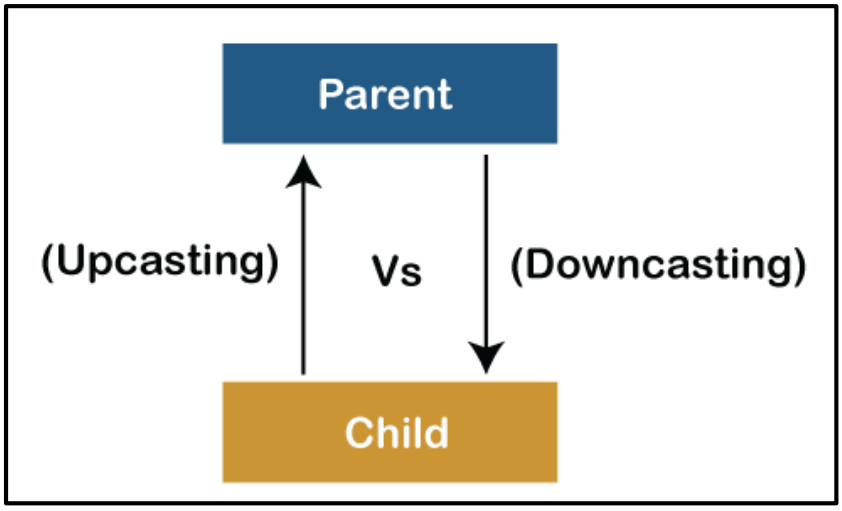
In the above program, notice the line

**String data = String.valueOf(num);**

Here, we have used the valueOf() method of the Java String class to convert the int type variable into a string.

## Object Casting:

In Java, the object can also be type casted like the datatypes. Parent and Child objects are two types of objects. So, there are two types of typecasting possible for an object, i.e., Child to Parent and Parent to Child or can say Upcasting and Down casting.




## Upcasting

Upcasting is a type of object typecasting in which a child object is type casted to a parent class object. By using the Upcasting, we can easily access the variables and methods of the parent class to the child class. Here, we don't access all the variables and the method. We access only some specified variables and methods of the child class. Upcasting is also known as Generalization and Widening.

Example:

```
class  Parent{
   void PrintData() {
      System.out.println("method of parent class");
   }
}

class Child extends Parent {
   void PrintData() {
      System.out.println("method of child class");
   }
}

public class Upcasting {
    public static void main(String args[]) {

      Parent obj1 = (Parent) new Child();
      Parent obj2 = (Parent) new Child();
      obj1.PrintData();
      obj2.PrintData();
   }
}
```

Output:

```
method of child class
method of child class
```

## Downcasting

In Java, we cannot assign a parent class reference object to the child class, but if we perform down casting, we will not get any compile-time error. However, when we run it, it throws the "ClassCastException". Now the point is if down casting is not possible in Java, then why is it allowed by the compiler? In Java, some scenarios allow us to perform down casting. Here, the subclass object is referred by the parent class.

```java
package com.mycompany.ooplab1;
class Parent {
    String name;

    // A method which prints the data of the parent class
    void showMessage()
    {
        System.out.println("Parent method is called");
    }
}

// Child class
class Child extends Parent {
    int age;

    // Performing overriding
    @Override
    void showMessage()
    {
        System.out.println("Child method is called");
    }
}
```

```java
public class downcasting {
    public static void main(String[] args)
    {
        Parent p = new Child();
        p.name = "Java Lab";

        // Performing Downcasting Implicitly
        //Child c = new Parent(); // it gives compile-time error

        // Performing Downcasting Explicitly
        Child c = (Child)p;

        c.age = 10;
        System.out.println(c.name);
        System.out.println(c.age);
        c.showMessage();
    }

}
```

## Inheritance:

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object.

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

**Inheritance represents the IS-A relationship which is also known as a parent-child relationship.**

Terms used in Inheritance

- Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- Super Class/Parent Class: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- Reusability: A mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class.

The syntax of Java Inheritance

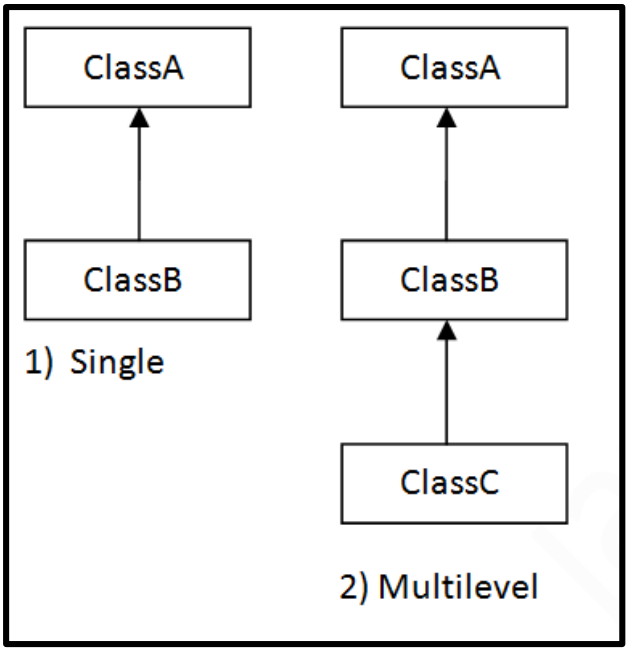
```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

The extends keyword indicates that you are making a new class that derives from an existing class.

**Types of inheritance in java**

On the basis of class, there can be four types of inheritance in java: single, multilevel, hierarchical and multiple inheritance.

In java programming, multiple and hybrid inheritance is supported through interface only.




**Single Inheritance Example**

When a class inherits another class, it is known as a single inheritance. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

**Using super to invoke constructors:**

To explicitly call the superclass constructor from the subclass constructor, we use super(). It's a special form of the super keyword.

super() can be used only inside the subclass constructor and must be the first statement.

The compiler can automatically call the no-argument constructor. However, it cannot call parameterized constructors.

If a parameterized constructor has to be called, we need to explicitly define it in the subclass constructor.

```java
class Animal {
  // default or no-arg constructor
  Animal() {
    System.out.println("I am an animal in default constructor");
  }
  // parameterized constructor
  Animal(String type) {
    System.out.println("Type: "+type);
  }
}

class Dog extends Animal {
  // default constructor
  Dog() {
    //calling default constructor
    //super();
    // calling parameterized constructor of the superclass
    super("Animal");

    System.out.println("I am a dog");
  }
}
class TestInheritance {
  public static void main(String[] args) {
    Dog dog1 = new Dog();
  }
}
```

Output:

```
Type: Animal
I am a dog
```

**Multilevel Inheritance Example**

When there is a chain of inheritance, it is known as multilevel inheritance. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

```java
package com.mycompany.ooplab1;
class Data
{
int length,breadth,height,area,volume;
public void input(int l, int b, int ht)
    {
        length = l;
        breadth = b;
        height = ht;
   }
}
class Area extends Data
{
    public int calculateArea()
    {
        area = length*breadth;
        return area;
    }
}
class Volume extends Area
{
    public int calculateVolume()
    {
        volume = area*height;
        return volume;
    }
}
```

```java
public class MultilevelInheritance {
    public static void main(String args[]){
        Volume v = new Volume();
        v.input(5,15,2);

    int ans = v.calculateArea();
      System.out.println("Area of rectangle = "+ans);
      System.out.println("Volume of rectangle = "+v.calculateVolume());
}}
```

Output:

```
Area of rectangle = 75
Volume of rectangle = 150
```